

Projet Planning Poker - Rapport

I - Introduction et contexte.....	1
II - Choix technique et architecture.....	2
III - Mise en place de l'intégration continue.....	2
IV - Manuel utilisateur et fonctionnalités.....	2

I - Introduction et contexte

Ce projet a été réalisé dans le cadre du cours de conception agile. L'objectif étant d'appliquer les concepts vu en CM tout en développant une application fonctionnelle.

Le planning poker est une méthode d'estimation que les équipes utilisent pour déterminer la complexité des tâches à réaliser. L'enjeu était donc de créer une application simulant ces techniques. Conformément aux consignes, l'application devait répondre à plusieurs exigences avec une liberté dans certains choix. C'est pourquoi nous avons choisi de développer l'application en mode de jeu local, tour par tour sur un même poste. L'application devait permettre à l'utilisateur de configurer sa partie (nombre de joueurs et paramètres), gérer le backlog des fonctionnalités à estimer, respecter les différentes règles de validations des votes, ainsi que d'assurer une sauvegarde des résultats à la fin de partie ou pendant.

Le projet a été réalisé en binôme, avec, à la base, la consigne de développer en Pair Programming. Dans la pratique, nous avons appliqué cette méthode de manière arrangée. Le développement principal s'est bien fait sur un seul ordinateur. Cependant, il nous est souvent arrivé de travailler en parallèle sur nos deux machines, l'un développait, l'autre corrigeait les petits problèmes mineurs d'affichages, consultait la documentation, ou modifiait un peu certaines parties du code en le factorisant notamment.

Même si cette organisation s'éloigne du Pair programming, elle s'est révélée efficace. Elle nous a permis d'optimiser notre temps en vue des délais et d'améliorer la qualité du code.

II - Choix technique et architecture

1. Langages

Pour ce projet, nous sommes partis sur du développement web “natif”. HTML pour la structure, CSS pour la mise en page et Javascript pour la logique. Nous avons choisi de nous concentrer sur une version locale où tous les joueurs votent sur le même poste, simplifiant l’architecture de l’application et évitant la gestion de websockets. Le JS étant directement exécuté par le navigateur, il offrait la solution la plus rapide pour une application locale. En utilisant du natif, nous avons pu nous concentrer sur la logique de jeu sans passer de temps à configurer un framework un peu plus lourd (Vue ou React). L’application ne nécessite pas de navigateur en particulier. Finalement, le planning poker étant une application assez simple, l’utilisation du *DOM* pour les mises à jours de l’interface et du *localStorage* pour les données, suffisait amplement et permettait une plus grande souplesse.

Nous n’avons pas utilisé de bibliothèque externe pour la logique de l’application afin de garder une séparation claire de chaque responsabilités des différentes parties de l’application. Seule exception avec la bibliothèque *Jest*, utilisée pour nos tests unitaires, ce qui nous a permis de vérifier la validité du code et d’en assurer la non régression.

2. Architecture de l’application

L’application ne repose pas sur une architecture en particulier, pas de strict MVC classique, qui aurait été bien plus pertinente à mettre en place si notre application était en ligne. Cependant nous avons séparé les différentes logiques pour la maintenabilité et la lisibilité du code.

Notre code est découpé en différents fichiers *.js* représentant chacun, une partie spécifique de l’application (logique métier, gestion de l’interface utilisateur, gestion des états de la partie, initialisation et affichages...). Cette organisation reste très simple et basique, mais garantissant que tout ne dépend pas forcément du DOM, notamment pour la réalisation de tests dans la chaîne d’intégration continue.

3. Modélisation des données

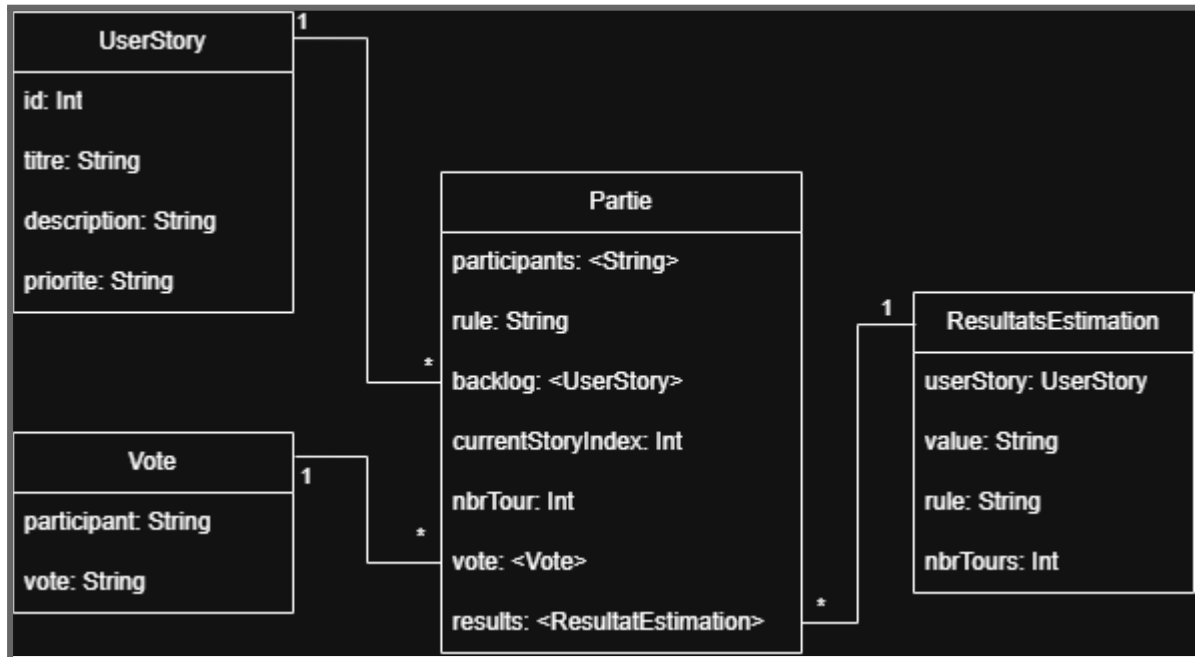
Afin d’éviter, comme précisé dans les consignes, de nous retrouver avec un “plat de spaghettis” ou plutôt une application difficile à maintenir et faire évoluer, nous nous sommes concentrés sur la modélisation d’objets stockés dans le *localStorage*, ou dans les fichiers *JSON*.

Le diagramme de classes ci-dessous montre que le jeu est centralisé sur la classe *Partie*, qui gère l’ensemble des données nécessaires au déroulement, au contexte et à la sauvegarde des résultats en cours et finaux.

La classe *UserStory* correspond à l'entité du backlog, basé sur le fichier *userstories.json* dans le dossier *assets/data* qui sert de jeu de données de test pour l'application.

La classe *Vote* correspond à un objet stocké dans le *localStorage* qui conserve chaque vote pour l'estimation finale. La classe *ResultatsEstimation* stocke dans un objet du *localStorage* les résultats de chaque *UserStory* afin de les retourner pour l'export final.

Diagramme de classe UML de l'application:



4. Gestion des données

La gestion des données repose sur l'utilisation de fichiers JSON (entrées et sorties) ainsi que du *localStorage* du navigateur pour la gestion de l'état de la session.

Le backlog est chargé depuis un fichier JSON (Ex: *assets/data/userstories.json*) :

```
{
  "backlog": [
    {
      "id": 1,
      "titre": "En tant qu'utilisateur, je veux pouvoir me connecter",
      "description": "Permettre à l'utilisateur de se connecter via email et mot de passe pour accéder à son espace.",
      "priorite": "haute"
    },
  ],
}
```

Comme l'application est locale, nous avons utilisé le *localStorage* du navigateur pour stocker l'état de la partie entre la page de configuration, la page de jeu et la page de résultats. Les données y sont persistées sous forme de chaînes JSON/texte.

Par exemple, *backlog* correspond à une liste de UserStory à estimer, *participants* est la liste des participants renseignées à la création de la partie, *rule* est la règle choisie, *currentStoryIndex* est l'index de la userStory en cours et *results* est la liste des estimations déjà validées. L'avantage de choix est surtout lié à la reprise de parti en cas de vote "expresso".

Deux types de JSON sont générés par différentes fonctions dans la gestion de partie. Le fichier de résultat final, ou la fin du backlog, est un fichier exportable lorsque toutes les fonctionnalités ont été estimées. Alors que le fichier de sauvegarde lié à la carte café, est exporté si tous les joueurs ou si le résultat correspond à cette carte. Il permet d'exporter des résultats intermédiaires et ainsi, d'avoir un fichier de reprise de partie qui peut être chargé sur la page d'accueil en relançant une partie à partir de ce dernier.

Les deux fichiers partagent une structure commune pour assurer la cohérence et la compatibilité lors de la reprise session :

Ce format garantit le suivi des estimations (qui a été estimé, à quelle valeur, selon quelle règle, combien de tours...), tout en permettant de recharger le backlog non estimé pour reprendre la partie.

```
{
  "sessionName": "Lyon 2",
  "participants": [
    "Lucas",
    "Mathéo",
    "Léo"
  ],
  "rule": "unanimite",
  "backlog": [
    {
      "id": 1,
      "titre": "En tant qu'utilisateur, je veux pouvoir me connecter",
      "description": "Permettre à l'utilisateur de se connecter via email et",
      "priorite": "haute",
      "etat": "estimée",
      "estimation": "8",
      "ruleUtilisee": "unanimite",
      "nombreTours": 3
    },
    {
      "id": 2,
      "titre": "En tant qu'utilisateur, je veux modifier mon profil",
      "description": "L'utilisateur doit pouvoir modifier son nom, son image",
      "priorite": "moyenne",
      "etat": "estimée",
      "estimation": "café",
      "ruleUtilisee": "unanimite",
      "nombreTours": 1
    }
  ]
}
```

III - Mise en place de l'intégration continue

1. Workflow

Pour répondre à l'exigence de mise en place de l'intégration continue (CI), nous avons choisi d'utiliser GitHub Actions, l'outil étant directement intégré à notre plateforme de gestion de code source, GitHub. Ce choix s'est fait de par sa simplicité de configuration et son intégration native avec notre dépôt, ce qui nous a permis d'éviter la gestion et la configuration d'un serveur dédié.

La configuration de l'intégration continue repose sur la création de deux fichiers de workflow au format YAML, stockés dans le répertoire `.github/workflows` à la racine du projet. Chaque fichier correspond à une pipeline dédiée à une tâche spécifique, notamment l'automatisation des tests et la génération de la documentation.

2. Tests Unitaires (tests.yml)

Cette première pipeline a pour mission d'assurer la non-régression de notre code JavaScript en automatisant l'exécution des tests unitaires que nous avons développés.

Le workflow est déclenché automatiquement lors de chaque push et de chaque pull request sur la branche main, ce qui nous permet d'assurer une vérification continue de la qualité du code.

```
name: Tests unitaire Javascript - Planning Poker
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up node
        uses: actions/setup-node@v4
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm run test
```

Le job principal, nommé test, s'exécute sur l'environnement ubuntu-latest et suit les étapes suivantes :

Checkout code :

Il s'agit de la première étape obligatoire de tout workflow qui a besoin d'accéder au code source d'un projet, notamment pour installer les dépendances puis exécuter les tests.

Le uses: actions/checkout@v4 exécute la commande git clone pour télécharger le code du dépôt GitHub sur la machine virtuelle temporaire (le runner) qui exécute notre job test.

Dans notre pipeline, cette étape permet de mettre le code à disposition pour l'étape Install dependencies ainsi que de s'assurer que l'étape Run tests dispose bien de tous les fichiers source et des fichiers de tests pour son bon déroulement.

Set up node :

Après avoir récupéré le code source, le runner, basé sur un système d'exploitation Ubuntu, ne dispose pas encore de l'environnement pour exécuter Nodejs.

L'action actions/setup-node@v4 a pour rôle de lui installer et de lui configurer son environnement : Elle télécharge la version de Nodejs spécifiée (ou une version par défaut si aucune n'est précisée) et l'installe sur son système, ce qui permet l'utilisation et la reconnaissance des commandes npm et node afin de pouvoir les utiliser dans les étapes suivantes du job.

Cette étape est clé puisqu'elle permet de mettre en place l'environnement nécessaire pour l'installation de la dépendance utilisée pour les tests.

Install dependencies :

Une fois l'environnement de développement Nodejs mis en place, le runner a besoin d'accéder à l'ensemble des dépendances et des outils externes nécessaires à l'exécution du projet.

Cette étape exécute la commande *npm install*. Son rôle est de lire le fichier de configurer des dépendances de notre projet, le package.json, afin d'installer toutes les bibliothèques et modules requis dans le répertoire node_modules du runner.

Run tests :

Cette étape représente l'objectif principal de ce workflow d'intégration continue : valider le bon fonctionnement du code.

Une fois que l'environnement Nodejs est configuré et que toutes les dépendances ont été installées, la commande npm run test est exécutée.

Cette commande lance le script de test unitaire qui est défini dans le fichier package.json du projet. Elle parcourt l'ensemble des tests qui se trouvent dans le fichier __test__/rules.test.js pour vérifier les différentes logiques et les éléments fonctionnels de l'application (validation des règles de vote, calculs de la moyenne/médiane, ...).

```
describe("Tests des Règles de Vote (rules.js)", () => {  
  // Test 1: Échec de l'unanimité (1er tour)  
  test("1. checkRule doit échouer si le 1er tour n'est pas unanime (votes différents)", () => {  
    const votes = createMockVotes(["8", "5", "8"]);  
    expect(checkRule(votes, "moyenne", true)).toBe(false);  
  });  
});
```

Cette pipeline a donc pour objectif, si tous les tests réussissent, de faire en sorte qu'aucune régression n'est possible. En effet, si un seul test échoue, l'étape échoue, le workflow est interrompu et signale immédiatement que le code n'est pas stable et empêche la fusion dans la branche main tant que les erreurs n'ont pas été corrigées.

3. Génération et déploiement de la documentation (documentation.yml)

Le second workflow est consacré à la génération automatique de la documentation technique de notre code et à son déploiement sur une URL.

Il est déclenché lors d'un push sur la branche main, ce qui permet de faire attention à ce que la documentation publiée soit toujours à jour selon la dernière version du code.

Pour les langages que nous avons utilisés, nous avons choisi JSDoc pour générer la documentation à partir des commentaires intégrés dans nos fichiers JavaScript.

```
name: Generate JSDoc Documentation - Planning Poker  
  
on:  
  push:  
    branches:  
      - main  
  
jobs:  
  build:  
    runs-on: ubuntu-latest  
  
    steps:  
      - name: Checkout repository  
        uses: actions/checkout@v4  
  
      - name: Setup Node  
        uses: actions/setup-node@v4  
  
      - name: Install JSDoc  
        run: npm install -g jsdoc  
  
      - name: Generate Documentation  
        run: jsdoc ./assets/js/*.js  
  
      - name: Deploy Documentation  
        uses: peaceiris/actions-gh-pages@v3  
        with:  
          github_token: ${{ secrets.GITHUB_TOKEN }}  
          publish_dir: ./out
```

Le job, nommé *build*, s'exécute sur le même environnement que le précédent, ubuntu-latest et suit les étapes suivantes :

Checkout repository, Setup Node :

Même rôle que pour le pipeline précédent, récupérer le code source et mettre en place l'environnement d'exécution *Nodejs*.

Install JsDoc :

Cette étape a pour objectif la mise à disposition de l'outil de génération de documentation choisir pour notre projet : *JSDoc*.

La commande *npm install -g jsdoc* est exécutée par le runner, elle utilise le gestionnaire de paquets de *Node* pour télécharger le module *JSDoc* et le *-g* assure qu'il est installé globalement sur son système. Cela permet de rendre la commande *jsdoc* directement accessible depuis le *PATH*.

Cette action prépare l'étape suivante en installant l'outil.

Generate Documentation :

Cette étape transforme le code source commenté en documentation lisible. Elle exécute la commande *jsdoc*, en lui fournissant le chemin de tous nos fichiers JavaScript présents dans le fichier : *./assets/js/*.

L'outil *JSDoc* analyse l'ensemble des fichiers JavaScript, extrait les commentaires et utilise ces informations pour générer un ensemble de fichiers HTML, CSS et JavaScript formant la documentation technique complète du projet. Par défaut, ces fichiers sont placés dans un répertoire nommé *./out*.

Deploy Documentation :

Cette dernière étape permet le déploiement de la documentation. Elle utilise une action qui permet l'automatisation de la publication sur GitHub Pages. Elle prend le contenu de l'HTML généré par *JSDoc*, se trouvant dans le répertoire *./out*, comme source à publier.

L'action utilise le jeton de sécurité interne de GitHub Actions (secrets.GITHUB_TOKEN) pour s'authentifier et effectuer les commandes nécessaires comme le push sur la branche de déploiement de GitHub Pages.

Ce processus automatise l'envoi des fichiers de la documentation vers le service GitHub Pages du dépôt, ce qui nous permet d'accéder à la documentation sur un URL dédiée : <https://lucas-ui.github.io/PlanningPoker/>.

Toute modification poussée sur la branche *main* déclenche à la fois la vérification par les tests unitaires mais également la mise à jour automatique de la documentation accessible en ligne.

IV - Manuel utilisateur et fonctionnalités

1. Installation et lancement

L'application Planning Poker a été développée en web "natif" sans nécessiter de backend. Elle est conçue pour une utilisation locale et ne demande l'installation d'aucun serveur.

Pour lancer le projet, il suffit de suivre les étapes de la documentation `README.md`, qui sont les suivantes :

- Clonage du dépôt : Permet de récupérer le code source grâce à Git.

```
git clone https://github.com/Lucas-ui/PlanningPoker.git  
cd PlanningPoker (cd est une commande Linux pour basculer entre les répertoires)
```

- Démarrage : Ouvrir le fichier principal `index.html` dans votre navigateur ou lancer l'extension Live Server.

Pour la vérification des tests unitaires mis en place, il est nécessaire d'installer les dépendances de développement définies dans le fichier `package.json`. Dans ce cas, à la racine du projet Planning Poker, il suffit d'effectuer `npm install`. Cette commande nécessite l'installation de Nodejs et du gestionnaire de paquets npm.

2. Modes de jeu

Nous avons implémenté le mode de jeu obligatoire (strict/unanimité) ainsi que tous les modes optionnels qui nous ont été présentés.

L'application supporte donc cinq modes de jeu, que l'utilisateur pourra sélectionner au moment de la création de la partie :

Mode de jeu	Règle de validation
Unanimité	Tous les joueurs doivent choisir la même carte pour que la fonctionnalité soit validée.
Majorité absolue	Plus de 50% des joueurs doivent avoir voté la même carte.
Majorité relative	Le vote retenu est celui ayant le plus grand nombre de votes.
Moyenne	La valeur d'estimation retenue est la moyenne des votes.

Médiane	La valeur d'estimation retenue est la médiane : valeur qui sépare la moitié inférieure et la moitié supérieure des votes.
---------	---

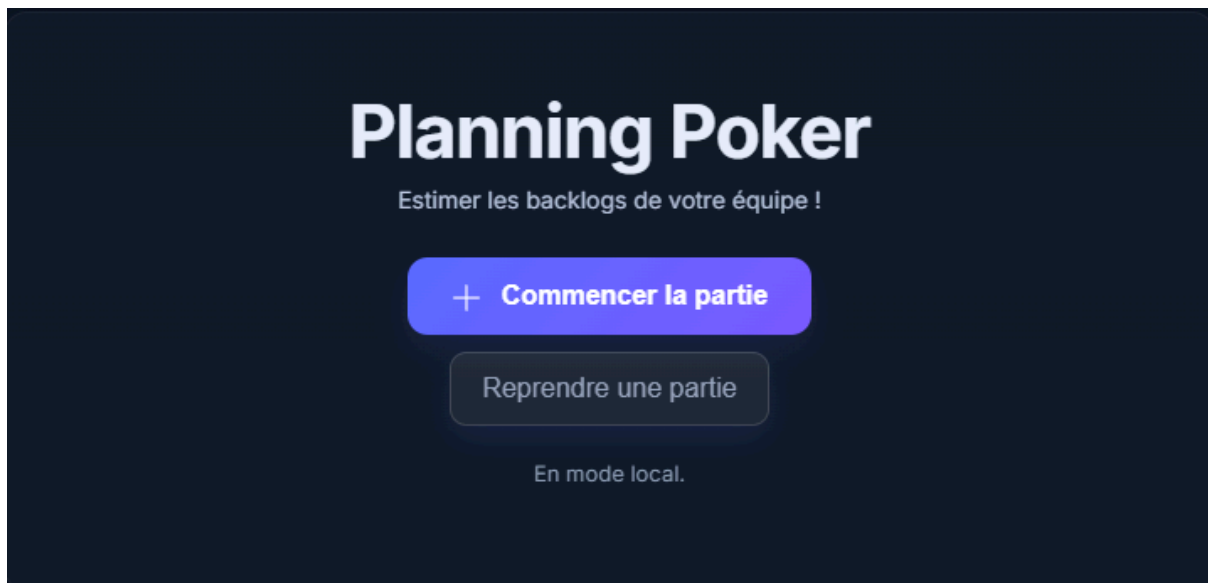
Une spécificité a été mise en place, celle-ci est valable pour tous les modes jeu, à l'exception de l'unanimité elle-même : Lors du premier tour, l'application vérifie automatiquement l'unanimité, cela force les joueurs à avoir un temps de discussion s'ils ne sont pas d'accord en premier lieu. Le mode de jeu sélectionné sera pris en compte pour les tours suivants.

3. Interface

L'application est conçue en trois vues principales (index.html, views/game.html, views/results.html), ce qui nous permet le flux de jeu suivant : Menu - Chargement JSON - Vote - Sauvegarde.

A) La page d'accueil

La page d'accueil (index.html) est le point d'entrée et le tableau de bord de l'application. Elle propose deux options : "Commencer la partie" ou "Reprendre une partie".



Le formulaire de création de partie apparaît lorsque l'utilisateur commence une partie. Il permet de définir un nom de session, un nombre de participants, l'importation d'un fichier JSON comportant le fichier JSON du backlog des User Stories à estimer et enfin la sélection d'une règle.

Lorsque la configuration de la session est terminée, une interface s'ouvre pour que chacun des participants rentrent son nom.

Configurer la session

Nom de la session

Nombre de participants

Importer une User Story (en JSON)

Sélectionner une règle

Configurer la session

Veuillez entrer le nom de chaque participant :

Participant 1

Participant 2

Participant 3

B) Le flux de vote

La page de jeu (views/game.html) est l'interface principale du Planning Poker. Elle affiche différentes informations concernant la User Story en cours, telles que son titre, sa priorité et sa description. Le joueur peut également percevoir à qui est le tour, et les multiples cartes qui permettent le vote. Les cartes sont cliquables pour enregistrer le choix du joueur en cours.

En tant qu'utilisateur, je veux pouvoir me connecter

HAUTE

Permettre à l'utilisateur de se connecter via email et mot de passe pour accéder à son espace.

Participant actuel : Lucas

0

1

2

3

5

8

13

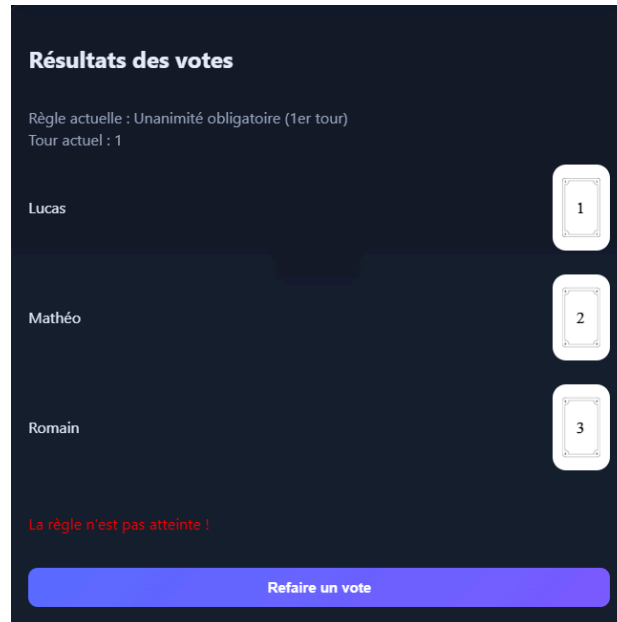
20

40

100

?

Dès lors que tous les joueurs ont voté, un bouton “Afficher les résultats” apparaît afin de passer à la révélation. Sur cette page, le joueur peut voir de nouveau les informations liées à la User Story mais également le résultat des votes. Un rappel de la règle en cours est affiché, le nombre de tours ainsi que le nom des participants et leur vote. Si la règle est validée, on passe à l’User Story suivante, sinon, un message “La règle n’est pas atteinte !” s’affiche et un nouveau tour de jeu se lance.



C) Sauvegarde et exportation JSON

Deux mécanismes d’exportation de données ont été mis en place :

- Si tous les joueurs votent la carte “Café”, un fichier JSON de sauvegarde est exporté. Ce fichier conserve l’état de la session et permet de reprendre la partie depuis le menu principal.
- Une fois le backlog entièrement estimé, l’utilisateur est dirigé vers la page de résultats (*results.html*). Cette page affiche le tableau des estimations. On retrouve le titre de toutes les Users Stories, le résultat, la règle et le nombre de tours. Un bouton permet d’exporter le fichier JSON contenant l’ensemble des résultats de la partie et un bouton de retour à l’accueil.

User Story	Résultats	Règle	Tours
En tant qu'utilisateur, je veux pouvoir me connecter	1	unanimité	2
En tant qu'utilisateur, je veux modifier mon profil	1	unanimité	2
En tant qu'admin, je veux gérer les utilisateurs	1	unanimité	1

Exporter en JSON Retour à l'accueil